EPAM INDIA

SDET INTERVIEW QUESTION BANK

PART 2 — ADDITIONAL QUESTIONS NOT IN YOUR FIRST PDF

53 questions • Real-world scenarios • Java • Selenium • Playwright • API • SQL • Test Strategy

Plain-English answers, practical examples, and dark code-editor blocks

Research approach

This companion guide was assembled from publicly accessible SDET interview guides, technical articles, GitHub question catalogs, and candidate/interviewer discussions on Reddit and YouTube. It focuses on additional topics beyond the 47 questions in Part 1. Public interview reports are anecdotal and cannot guarantee that a company will ask a specific question.

Study method: explain the concept in your own words, write the small code example, then describe one real framework use case and one trade-off.

Contents

Java coding & language internals — questions 1–15

Selenium & framework engineering — questions 16–24

Playwright — questions 25–30

API testing & distributed systems — questions 31–40

SQL, test design & senior scenarios — questions 41–53

Total questions in this companion guide: 53

SECTION 1 — JAVA CODING & LANGUAGE INTERNALS

Q01. Reverse a linked list in Java

Layman explanation: Change each node's next pointer so it points to the previous node; keep track of previous, current, and next nodes.

Example: For 1 → 2 → 3, the result is 3 → 2 → 1.

CODE / ILLUSTRATION

```
class Node {
    int value;
    Node next;
    Node(int value) { this.value = value; }
}
static Node reverse(Node head) {
    Node prev = null, current = head;
    while (current != null) {
        Node next = current.next;
        current.next = prev;
        prev = current;
        current = next;
    }
    return prev;
}
```

Interview tip: Time O(n), extra space O(1). Explain the pointer changes carefully.

Q02. Compress repeated characters in a string

Layman explanation: Count consecutive repeated characters and append each character followed by its count.

Example: Input aaaabbbbbcc becomes a4b5c2.

CODE / ILLUSTRATION

```
static String compress(String s) {
    if (s == null || s.isEmpty()) return s;
    StringBuilder out = new StringBuilder();
    int count = 1;
    for (int i = 1; i <= s.length(); i++) {
        if (i < s.length() && s.charAt(i) == s.charAt(i - 1)) {
            count++;
        } else {
            out.append(s.charAt(i - 1)).append(count);
            count = 1;
        }
    }
    return out.toString();
}
```

Interview tip: Clarify whether counts of 1 should be included and whether compression must be shorter than the input.

Q03. Find the first and last occurrence in a sorted array

Layman explanation: Use binary search twice: one search keeps looking left after a match; the other keeps looking right.

Example: For [1, 2, 2, 2, 5], target 2 gives first index 1 and last index 3.

CODE / ILLUSTRATION

```
static int first(int[] a, int target) {
    int lo = 0, hi = a.length - 1, ans = -1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] >= target) hi = mid - 1;
        else lo = mid + 1;
        if (a[mid] == target) ans = mid;
    }
    return ans;
}
```

Interview tip: The shown method finds the first occurrence. Implement the last occurrence symmetrically by moving right after a match. Complexity O(log n).

Q04. How does HashMap work internally?

Layman explanation: HashMap uses a key's hash to choose a bucket, then uses equality to identify the correct key within that bucket. Collisions can share a bucket; modern Java can treeify heavily-collided buckets under certain conditions.

Example: A good hashCode spreads keys across buckets; equals identifies whether two keys represent the same logical key.

Interview tip: Important: equal objects must have equal hash codes. HashMap is not inherently thread-safe.

Q05. Difference between == and equals()

Layman explanation: For objects, == compares whether two references point to the same object; equals() compares logical equality when the class implements it appropriately. For primitives, == compares values.

Example: Two different String objects can contain the same text: == may be false while equals() is true.

CODE / ILLUSTRATION

```
String a = new String("EPAM");
String b = new String("EPAM");
System.out.println(a == b);      // false
System.out.println(a.equals(b)); // true
```

Interview tip: Do not assume every class overrides equals(); Object's default implementation behaves like reference identity.

Q06. Why override hashCode() when overriding equals()?

Layman explanation: Hash-based collections use hashCode() to find a bucket and equals() to check a matching key. If equal objects have different hash codes, HashMap/HashSet may fail to find duplicates correctly.

Example: Two Employee objects with the same employee ID should be equal and return the same hash code if ID defines equality.

CODE / ILLUSTRATION

```
@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (!(obj instanceof Employee)) return false;
    Employee other = (Employee) obj;
    return id == other.id;
}
@Override
public int hashCode() {
    return Integer.hashCode(id);
}
```

Interview tip: Keep equality based on stable fields, especially if objects are used as map keys.

Q07. String vs StringBuilder vs StringBuffer

Layman explanation: String is immutable; StringBuilder is mutable and usually preferred for repeated string changes in one thread; StringBuffer synchronizes its methods for thread-safe individual operations.

Example: Use StringBuilder in loops that build a report or request payload.

CODE / ILLUSTRATION

```
StringBuilder result = new StringBuilder();
for (String part : parts) {
    result.append(part).append(",");
}
String csv = result.toString();
```

Interview tip: StringBuffer's synchronized methods do not automatically make a multi-step business operation atomic.

Q08. Abstract class vs interface

Layman explanation: An abstract class can hold shared state and constructors; an interface defines a contract and can also provide default/static methods. A class can extend one class but implement multiple interfaces.

Example: Use an interface for interchangeable browser-independent capabilities; use an abstract base class when related classes genuinely share implementation/state.

Interview tip: Explain the design need instead of saying one is always better.

Q09. Explain SOLID with automation examples

Layman explanation: SOLID is a set of design principles: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion.

Example: Example: a page class should perform page actions, while reporting, configuration, and driver creation belong in separate components.

Interview tip: Tie each principle to reduced coupling, easier change, and simpler tests; avoid building unnecessary abstractions.

Q10. What is immutability and how do you create an immutable class?

Layman explanation: An immutable object cannot change its observable state after construction. Use private final fields, validate/copy inputs, avoid setters, and do not expose mutable internals.

Example: An immutable configuration object is safer to share between parallel tests.

CODE / ILLUSTRATION

```
public final class EnvironmentConfig {
    private final String baseUrl;
    public EnvironmentConfig(String baseUrl) {
        this.baseUrl = baseUrl;
    }
    public String getBaseUrl() { return baseUrl; }
}
```

Interview tip: If a field is a mutable list or date, return defensive copies or use immutable collection/value types.

Q11. What is the difference between volatile and synchronized?

Layman explanation: volatile makes reads/writes of a variable visible across threads but does not make compound operations atomic. synchronized provides mutual exclusion and visibility around a critical section.

Example: count++ is not made atomic merely by declaring count volatile.

CODE / ILLUSTRATION

```
private volatile boolean stopped = false;

// For atomic counters use:
private final AtomicInteger count = new AtomicInteger();
count.incrementAndGet();
```

Interview tip: Choose based on the shared-state operation; explain visibility versus atomicity.

Q12. What is a deadlock and how can you prevent it?

Layman explanation: Deadlock occurs when threads wait forever for locks held by one another. Prevent it by acquiring locks in a consistent order, minimising nested locks, and using timed lock attempts where appropriate.

Example: Thread A holds lock X and waits for Y while Thread B holds Y and waits for X.

Interview tip: Thread dumps help identify threads and locks involved. Do not use arbitrary sleeps as a fix.

Q13. ExecutorService: why use it?

Layman explanation: ExecutorService manages a pool of worker threads so tasks can be submitted without manually creating and managing a thread for every task.

Example: Useful for controlled parallel API checks or background tasks; in UI automation, also consider browser and test-framework thread safety.

CODE / ILLUSTRATION

```
ExecutorService pool = Executors.newFixedThreadPool(4);
try {
    Future<Integer> result = pool.submit(() -> 2 + 3);
    System.out.println(result.get());
} finally {
    pool.shutdown();
}
```

Interview tip: Always shut down executors and handle task failures/timeouts.

Q14. What is the difference between overloading and overriding?

Layman explanation: Overloading uses the same method name with different parameter lists; overriding changes an inherited method's implementation with a compatible signature.

Example: Overloading is resolved mainly at compile time; overridden instance-method dispatch is dynamic at runtime.

CODE / ILLUSTRATION

```
void login(String user) { }
void login(String user, String password) { } // overload

@Override
public String toString() { return "Employee"; } // override
```

Interview tip: Changing only the return type is not sufficient for method overloading.

Q15. What are Java access modifiers?

Layman explanation: public is accessible broadly; protected is accessible in the package and through subclass rules; package-private has no modifier; private is accessible only within the declaring class.

Example: Keep page locators private and expose meaningful page actions publicly.

Interview tip: Be precise about protected access across packages and inheritance.

SECTION 2 — SELENIUM & FRAMEWORK ENGINEERING

Q16. How do you handle StaleElementReferenceException?

Layman explanation: The element reference points to a DOM element that has been replaced or detached. Re-locate it after the page/component updates and wait for the relevant state.

Example: After a React page refreshes a table row, reuse a locator rather than an old WebElement reference.

CODE / ILLUSTRATION

```
By saveButton = By.id("save");
WebDriverWait wait = new WebDriverWait(
    driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.elementToBeClickable(saveButton)).click();
```

Interview tip: Do not wrap every action in unlimited retries; determine why the DOM changes and use a stable locator.

Q17. Implicit wait vs explicit wait

Layman explanation: Implicit wait applies globally to element lookup; explicit wait waits for a specific condition. Avoid mixing them because combined timing can become confusing.

Example: Use explicit waits for a button to become clickable or a result to appear.

CODE / ILLUSTRATION

```
WebDriverWait wait = new WebDriverWait(
    driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.visibilityOfElementLocated(
    By.id("result")));
```

Interview tip: Condition-based waits are usually more precise than Thread.sleep.

Q18. How do you handle JavaScript alerts in Selenium?

Layman explanation: Switch to the browser alert and accept, dismiss, read text, or enter a prompt value.

Example: For a confirmation dialog, verify the message and choose the expected action.

CODE / ILLUSTRATION

```
Alert alert = driver.switchTo().alert();
String message = alert.getText();
alert.accept(); // or alert.dismiss()
```

Interview tip: Wait for alert presence when it appears asynchronously; browser alerts are different from HTML modal dialogs.

Q19. How do you upload a file using Selenium?

Layman explanation: For a standard input type=file, send the absolute file path to the input element rather than clicking the operating-system file chooser.

Example: Use a stable locator for the file input and verify the uploaded filename/status.

CODE / ILLUSTRATION

```
Path file = Paths.get("src/test/resources/sample.pdf")
                 .toAbsolutePath();
driver.findElement(By.cssSelector("input[type='file']"))
      .sendKeys(file.toString());
```

Interview tip: Custom upload widgets may require a different approach. Avoid hard-coded machine-specific paths.

Q20. How do you take a screenshot on test failure?

Layman explanation: Capture a screenshot when a test fails and attach it to the report along with URL, browser, logs, and test name.

Example: A screenshot helps show whether the page was blank, a modal covered the page, or the wrong state appeared.

CODE / ILLUSTRATION

```
File src = ((TakesScreenshot) driver)
    .getScreenshotAs(OutputType.FILE);
Files.copy(src.toPath(),
    Paths.get("target/failure.png"),
    StandardCopyOption.REPLACE_EXISTING);
```

Interview tip: Use unique filenames for parallel runs and ensure the screenshot is attached before the browser closes.

Q21. How does Selenium Grid help?

Layman explanation: Grid routes WebDriver commands to remote browser nodes so tests can run across browsers, operating systems, and machines, including parallel execution.

Example: A CI job can run Chrome and Firefox tests on separate nodes.

Interview tip: Discuss hub/router, nodes, capabilities, scaling, and infrastructure observability at a high level.

Q22. How do you choose CSS selector vs XPath?

Layman explanation: CSS is concise for IDs/classes/attributes and hierarchy; XPath supports text-based matching and navigating relationships such as parent/sibling axes.

Example: Prefer stable test IDs or semantic attributes; use XPath when relationships genuinely make it clearer.

Interview tip: Avoid brittle absolute XPath and selectors tied to layout or generated classes.

Q23. How would you design a UI automation framework from scratch?

Layman explanation: Start with requirements and constraints, then define test runner, driver/browser lifecycle, page/component objects, config, test data, assertions, logging/reporting, parallel isolation, CI, and failure diagnostics.

Example: For a login flow, keep locators/actions in LoginPage, test intent in the test, and driver creation in a managed fixture.

Interview tip: Explain trade-offs, folder structure, environment configuration, secrets, and how the framework remains maintainable as it grows.

Q24. Playwright: Browser vs BrowserContext vs Page

Layman explanation: Browser is the browser process; BrowserContext is an isolated session with its own cookies/storage; Page is a tab within a context.

Example: Create separate contexts to isolate users or roles while reusing a browser process.

CODE / ILLUSTRATION

```
const browser = await chromium.launch();
const context = await browser.newContext();
const page = await context.newPage();
await page.goto("https://example.com");
await browser.close();
```

Interview tip: Use isolated contexts to reduce cross-test contamination; APIs shown are JavaScript/TypeScript Playwright style.

SECTION 3 — PLAYWRIGHT

Q25. Why does Playwright auto-wait?

Layman explanation: Playwright actions wait for relevant actionability conditions—such as visibility, stability, enabled state, and receiving events—before interacting, reducing many timing flakes.

Example: A locator.click() waits for the element to become actionable rather than requiring a fixed sleep.

Interview tip: Auto-wait does not prove the business operation succeeded; assert the resulting state.

Q26. What is a Playwright fixture?

Layman explanation: A fixture provides reusable setup/teardown and dependencies for tests, such as a page, authenticated context, API client, or test data.

Example: A custom fixture can create a logged-in page and clean up data after a test.

CODE / ILLUSTRATION

```
import { test, expect } from "@playwright/test";

test("home page", async ({ page }) => {
  await page.goto("https://example.com");
  await expect(page).toHaveTitle(/Example/);
});
```

Interview tip: Prefer built-in fixtures and scoped setup over global shared mutable state.

Q27. How do you reuse authentication in Playwright?

Layman explanation: Log in once and save browser storage state, then create test contexts from that state when safe and appropriate.

Example: Useful when many tests need an authenticated session; do not share state if tests mutate the same account concurrently.

CODE / ILLUSTRATION

```
// Setup project: save state after login
await page.context().storageState({ path: "state.json" });

// Later
const context = await browser.newContext({
  storageState: "state.json"
});
```

Interview tip: Protect storage-state files because they may contain session credentials; keep them out of source control.

Q28. How do you intercept or mock a network request in Playwright?

Layman explanation: Use route interception to inspect, modify, abort, or fulfil a request. This isolates UI tests from unstable third-party services.

Example: Mock a recommendation API to return deterministic data while testing how the page renders it.

CODE / ILLUSTRATION

```
await page.route("**/api/recommendations", async route => {
  await route.fulfill({
    status: 200,
    contentType: "application/json",
    body: JSON.stringify([{ id: 1, name: "Sample" }])
  });
});
```

Interview tip: Use real services in integration/contract tests too; mocks cannot prove the real service contract is correct.

Q29. How do you debug a failing Playwright test?

Layman explanation: Use the error output, trace viewer, screenshots, video where configured, browser console logs, and network activity to find the first incorrect state.

Example: A trace can reveal that a click occurred but the expected response never arrived.

Interview tip: Reproduce locally, compare CI/local environments, and fix the cause rather than increasing timeouts blindly.

Q30. How do you run Playwright tests in parallel safely?

Layman explanation: Use independent tests, isolated browser contexts, unique test data, and controlled worker counts. Avoid shared accounts or records that tests mutate concurrently.

Example: Generate a unique email/order ID per worker and clean up created data.

Interview tip: Parallelism is a resource and data-isolation problem as much as a runner setting.

SECTION 4 — API TESTING & DISTRIBUTED SYSTEMS

Q31. How would you test an API beyond status code 200?

Layman explanation: Validate schema and required fields, business rules, headers, authentication, negative inputs, boundary values, data persistence, response time, and side effects.

Example: For create-user, verify 201, response ID, stored data, duplicate-email behaviour, and invalid payload responses.

Interview tip: Choose checks from the API contract and risk; avoid asserting implementation details that clients do not depend on.

Q32. Difference between 401 and 403

Layman explanation: 401 means authentication is missing or invalid; 403 means the server understood the identity but refuses permission for the resource.

Example: An expired token often produces 401; a valid user lacking admin permission often receives 403.

Interview tip: Actual APIs can differ in implementation details, so verify the published contract.

Q33. What is contract testing?

Layman explanation: Contract testing checks that a provider and its consumers agree on request/response expectations without requiring every test to run a full end-to-end system.

Example: A consumer expects GET /users/{id} to return an ID and name; provider verification checks that response contract.

Interview tip: Tools include Pact and schema-based validation; distinguish contract tests from broad end-to-end tests.

Q34. What is idempotency in APIs?

Layman explanation: An operation is idempotent if repeating the same request has the same intended effect as performing it once. PUT and DELETE are defined as idempotent by HTTP semantics, although responses can differ; POST is not generally idempotent.

Example: Retrying a payment POST after a timeout can create duplicate charges unless the API supports an idempotency key.

Interview tip: Test repeated requests, timeout/retry behaviour, and side effects—not just response codes.

Q35. How do you test pagination?

Layman explanation: Verify page/limit or cursor behaviour, ordering, no duplicates or missing records across pages, empty/final pages, invalid parameters, and consistency while data changes.

Example: Fetch pages 1–3 and verify the combined IDs contain each expected record exactly once.

Interview tip: Check whether the API uses offset or cursor pagination and whether the order is deterministic.

Q36. How do you test rate limiting?

Layman explanation: Send requests at a controlled rate and verify the documented limit, response status, retry guidance, and recovery after the window resets.

Example: A client exceeding a quota may receive 429 Too Many Requests and a Retry-After header.

Interview tip: Do not run uncontrolled load against production; coordinate test limits and environment.

Q37. How do you test eventual consistency?

Layman explanation: After a write, related reads or downstream services may not update immediately. Poll a bounded condition with a deadline rather than assuming immediate consistency.

Example: After creating an order, wait until the search index exposes it, then assert its fields.

Interview tip: Use bounded polling, clear timeout diagnostics, and distinguish delayed consistency from a genuine failure.

Q38. How do you mock a third-party API that is unavailable?

Layman explanation: Replace the dependency at the HTTP boundary with a deterministic stub, or mock an internal dependency in unit tests; separately test the real integration when available.

Example: WireMock can simulate a payment provider returning success, timeout, 500, or malformed JSON.

Interview tip: Cover timeout, connection refusal, slow response, retries, fallback, and circuit-breaker behaviour where applicable.

Q39. How do you validate JSON schema in REST Assured?

Layman explanation: Compare the response JSON structure and data types against a schema so missing fields, unexpected types, and contract drift are detected.

Example: A user ID expected as a number should fail validation if the service starts returning an incompatible object.

CODE / ILLUSTRATION

```
given()
.when().get("/users/1")
.then()
    .statusCode(200)
    .body(matchesJsonSchemaInClasspath("user-schema.json"));
```

Interview tip: Use the schema matcher dependency compatible with the REST Assured version; also assert important business values.

Q40. How do you validate database changes after an API call?

Layman explanation: Call the API, then query the database using a safe test connection to verify persisted state and transactional effects; clean up test data.

Example: After POST /users returns 201, query the test database by the returned ID and verify the stored email.

Interview tip: Avoid coupling every API test to database internals; use DB assertions where persistence is part of the test objective.

SECTION 5 — SQL, TEST DESIGN & SENIOR SCENARIOS

Q41. Second-highest salary using SQL

Layman explanation: Find the second distinct salary by selecting the maximum salary lower than the maximum salary, or use a ranking window function.

Example: If salaries are 100, 90, 90, 80, the second distinct salary is 90.

CODE / ILLUSTRATION

```
SELECT MAX(salary) AS second_highest
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);
```

Interview tip: Clarify whether the interviewer wants second distinct salary or the second row after sorting.

Q42. Find duplicate values using SQL

Layman explanation: Group rows by the candidate key and keep groups whose count is greater than one.

Example: Find duplicate email addresses in a Users table.

CODE / ILLUSTRATION

```
SELECT email, COUNT(*) AS occurrences
FROM Users
GROUP BY email
HAVING COUNT(*) > 1;
```

Interview tip: Ask whether NULL values should count as duplicates and whether case/whitespace should be normalised.

Q43. What are SQL window functions?

Layman explanation: Window functions calculate values across related rows without collapsing each group into one row. Common examples include ROW_NUMBER, RANK, DENSE_RANK, and running totals.

Example: Rank employees by salary within each department.

CODE / ILLUSTRATION

```
SELECT name, department_id, salary,
       DENSE_RANK() OVER (
         PARTITION BY department_id
         ORDER BY salary DESC
       ) AS salary_rank
FROM Employee;
```

Interview tip: RANK leaves gaps after ties; DENSE_RANK does not; ROW_NUMBER assigns a unique sequence.

Q44. How would you test a login page?

Layman explanation: Cover valid and invalid credentials, empty fields, lockout/rate limiting, password masking, keyboard/accessibility behaviour, session creation, logout, redirects, and security controls.

Example: Test correct login, wrong password, unknown user, locked user, expired session, and repeated attempts.

Interview tip: Include both UI assertions and API/security checks where appropriate; never expose real credentials in logs.

Q45. How would you test a payment workflow?

Layman explanation: Test successful payment, decline, timeout, duplicate submission, retry, cancellation/refund, currency/amount boundaries, idempotency, reconciliation, and audit trail.

Example: Simulate provider timeout after request acceptance and verify retry does not charge twice.

Interview tip: Use sandbox accounts and mocks for deterministic failures; verify financial side effects and reconciliation.

Q46. What is the difference between smoke, sanity, and regression testing?

Layman explanation: Smoke checks whether a build is stable enough for further testing; sanity focuses on a narrow change or fix; regression checks whether existing functionality still works after changes.

Example: After a deployment, run smoke tests first, focused sanity tests for a hotfix, and broader regression based on risk.

Interview tip: Definitions can vary slightly by organisation; explain the purpose and scope.

Q47. How do you prioritise automation candidates?

Layman explanation: Prioritise stable, repeatable, high-risk or high-frequency scenarios with clear expected results and useful data setup; avoid automating unstable features before they are testable.

Example: A critical payment regression run nightly may be higher priority than a rarely used, rapidly changing marketing animation.

Interview tip: Consider maintenance cost, execution time, failure diagnosis, and value—not simply the number of automated tests.

Q48. How do you measure automation quality?

Layman explanation: Track meaningful signals such as pass rate, flaky rate, failure diagnosis time, escaped defects, coverage of critical risks, suite duration, and maintenance effort.

Example: If a suite has a high pass rate but frequently needs reruns, its flakiness and trustworthiness need attention.

Interview tip: Avoid using raw test count as the only success metric; metrics should guide improvement.

Q49. What is a test data strategy for parallel tests?

Layman explanation: Create deterministic, isolated data per test or worker, use unique identifiers, provision and clean up records, and avoid shared mutable accounts when tests can conflict.

Example: Generate a unique order ID for each run and tag created data with a run ID for cleanup.

Interview tip: Design for repeatability and failure recovery; do not rely on tests executing in a specific order.

Q50. What should an SDET system-design answer include?

Layman explanation: Clarify requirements and scale, draw the components and data flow, explain test layers and interfaces, isolate environments/data, define reporting and observability, and discuss failure modes and trade-offs.

Example: For a file-download service, cover API validation, storage, asynchronous processing, access control, download URL expiry, and end-to-end checks.

Interview tip: Ask clarifying questions before choosing tools. Explain how the design supports debugging and safe parallel execution.

Q51. How do you test asynchronous jobs or queues?

Layman explanation: Trigger the operation, then poll for a bounded final state and verify retries, duplicate delivery, ordering requirements, dead-letter behaviour, and observability.

Example: Submit a report-generation job and wait until status becomes COMPLETED before checking the artifact.

Interview tip: Avoid fixed sleeps; use polling with a timeout and useful diagnostics.

Q52. How do you handle secrets in automation pipelines?

Layman explanation: Store secrets in a credential manager or CI secret store, inject them at runtime, restrict permissions, rotate them, and mask them in logs.

Example: Jenkins credentials binding supplies an API token without committing it to Git.

Interview tip: Never put secrets in source code, test reports, screenshots, or command-line output.

Q53. How do you test accessibility in automation?

Layman explanation: Combine automated checks with keyboard and assistive-technology testing; check semantic roles, accessible names, focus order, contrast, and error announcements.

Example: Use role-based locators and an accessibility scanner, then manually verify critical keyboard flows.

Interview tip: Automated scanners find only a subset of accessibility problems; they do not replace human evaluation.

Sources consulted

Sources consulted

• **Reddit — SDET interview guide/help, candidate-reported coding questions (2026)**
https://www.reddit.com/r/QualityAssurance/comments/1su9po0/sdet-interview-guidehelp/

• **Reddit — Senior QA interview experience and automation scenarios**
https://www.reddit.com/r/QualityAssurance/comments/1hg8u8v

• **Reddit — Senior SDET system design and automation preparation**
https://www.reddit.com/r/QualityAssurance/comments/1s4765x/how-to-prepare-for-sr-sdet-at-workday-india/

• **GitHub — SDET Interview Question Catalog**
https://github.com/Test-Architect/sdet-interview-question-catalog

• **JavaInUse — SDET frequently asked interview questions**
https://www.javainuse.com/misc/sdet

• **SoftwareTestPilot — SDET interview questions and answers (2026)**
https://softwaretestpilot.com/sdet-interview-questions

• **ScrollTest — JavaScript, Playwright and Selenium SDET interview guide**
https://scrolltest.com/sdet-interview-guide-javascript-playwright-selenium-questions-answers/

• **ClapAssist — API testing interview questions (2026)**
https://clapassist.com/tools/interview-questions/api-testing/

• **Automation Juice — Selenium and browser automation interview questions**
https://juice.upcode.in/interview-questions/selenium

• **YouTube — Senior QA & SDET interview questions (2026)**
https://www.youtube.com/watch?v=CImIdK9AHAo

Note: This is a curated preparation resource, not a verbatim reproduction of any one source. Question wording and examples have been summarised or newly authored for study. Always verify framework-specific APIs against the versions used in your project.